

6-3 · Producer / Consumer 애플리케이션 · 실습 2.5H

Producer / Consumer 코드와 주문 이벤트 파이프라인

파이썬으로 양쪽을 만들고, 유실·중복·LAG를 직접

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

2.5시간 뒤에는

- ① confluent-kafka 로 주문 이벤트 30건을 발행하고 delivery report 를 본다
- ② flush 를 빼고 1000건을 보내 유실을 재현한다
- ③ poll 루프 Consumer 로 고객별 누적 금액을 집계한다
- ④ 이어읽기·처음부터 읽기·수동 커밋·LAG 을 각각 확인한다

6-2 의 docker compose 가 떠 있고 orders 토픽(파티션 3)이 있어야 시작할 수 있습니다

일곱 단계 — ~/data-eng/week6 한 폴더에서 끝까지

호스트에서 붙으므로 bootstrap.servers 는 localhost:9092 — 6-2 의 EXTERNAL 리스너가 여기서 쓰입니다

1. 환경 준비	2. Producer	3. 일부러 실패	4·5. Consumer	6·7. 커밋·LAG
.venv 활성화 confluent-kafka	주문 30건 키 = 고객	flush 없이 1000건	누적 집계 이어읽기	수동 커밋 LAG 관찰

1단계

환경 준비 — 버전이 찍히면 준비 완료

위치	<code>cd ~/data-eng/week6</code>
kafka 확인	<code>docker compose ps</code>
가상환경	<code>source .venv/bin/activate</code>
설치	<code>pip install confluent-kafka</code>
확인	<code>python -c "import confluent_kafka; print(confluent_kafka.version())"</code>

이번 주 Python 클라이언트는 **confluent-kafka** 로 통일 — Python 3.10+ 가상환경에서

producer.py 핵심 줄 — 주문 이벤트를 1초에 하나씩

줄	뜻
<code>Producer({"bootstrap.servers": ...})</code>	localhost:9092 로 연결
<code>def on_delivery(err, msg)</code>	건별 성공·실패 콜백
<code>order = {order_id, customer, amount}</code>	JSON 주문 이벤트
<code>p.produce("orders", key=customer, ...)</code>	고객 ID 를 키로 — 같은 파티션에
<code>p.poll(0)</code>	쌓인 콜백 처리
<code>finally: p.flush()</code>	버퍼를 비우고 종료

python producer.py — 30건이 1초 간격으로 나간다

확인: 「전송 완료 topic=orders partition=N offset=M」 30줄 — 같은 customer 키는 항상 같은 partition 번호

flush 를 빼고 1000건 — 이론의 경고를 직접

sleep 을 없애고 flush 도 뺀 producer_noflush.py 로 orders-test 토픽에 보냅니다

토픽 생성	producer_noflush	sleep 3	GetOffsetShell	flush 넣고 재실행
orders-test 파티션 1	1000건 produce flush 없음	잠시 기다린 뒤	실제 쌓인 개수 확인	정확히 1000 비교

「보냈다고 생각한 건수」와 실제 쌓인 개수

flush() 없음

프로그램이 먼저 끝나 버퍼에 남은 것이 사라진다

- 보냈다고 생각한 건수: 1000
- GetOffsetShell 이 준 수가 더 적으면 유실 재현
- 환경에 따라 다 들어갈 수도 있다

VS

flush() 있음

버퍼를 다 보낼 때까지 기다린 뒤 끝난다

- 정확히 1000
- 1000 이 나와도 「안전하다」는 뜻은 아니다
- 메시지가 크거나 네트워크가 느리면 쉽게 유실

consumer.py 핵심 줄 — poll 루프와 고객별 누적

줄	뜻
"group.id": "order-agg"	이 Consumer 가 속한 그룹
"auto.offset.reset": "earliest"	새 그룹은 처음부터
c.subscribe(["orders"])	토픽 구독
msg = c.poll(1.0)	없으면 1초 기다렸다 None
if msg.error(): ...	오류를 조용히 넘기지 않는다
json.loads(msg.value().decode("utf-8"))	JSON 역직렬화
finally: c.close()	그룹 정상 탈퇴 + 오프셋 정리

터미널 1 python consumer.py · 터미널 2 python producer.py — 주문이 실시간으로 찍히고 누적 금액이 늘어나면 성공

이어읽기 · 처음부터 · 수동 커밋 · LAG

1 같은 그룹으로 다시 켜면 — 이어읽기
오프셋이 커밋되어 있어 이미 집계한 메시지를 다시 읽지 않는다

2 group.id 를 order-agg-2 로 — 처음부터
earliest 가 새 그룹에만 적용된다는 이론을 확인

3 enable.auto.commit False + c.commit(msg)
amount 가 5의 배수면 예외 — 실패한 메시지는 커밋되지 않아 재시작 때 다시 읽는다

LAG 관찰

꺼 두면 쌓이고
켜면 0

kafka-consumer-groups.sh --describe --group order-agg · 수동 커밋 코드는 실패 메시지에서 계속 막힌다 — 실무는 재시도 뒤 DLQ

이 2.5시간이 끝날 때 확인할 것

- 1 「전송 완료 ... partition=N offset=M」 30줄, 같은 키는 같은 partition
- 2 flush 없이 보낸 orders-test 는 1000 보다 적을 수 있다
- 3 Consumer 터미널에 고객별 누적 금액이 늘어난다
- 4 같은 그룹은 새 메시지만, 다른 그룹은 전부 다시 읽는다
- 5 수동 커밋에서 실패한 메시지는 재시작 때 다시 읽힌다
- 6 Consumer 를 끄면 LAG 이 쌓이고, 켜면 전 파티션 0 이 된다

미니 과제: consumer.py 둘 동시 실행 · acks all 추가 · 집계를 파일이나 SQLite 에 저장

6-3 · 실습 2.5H 마무리

다음은 정리·회고 (0.5H)

코드 리뷰 표로 빠뜨리기 쉬운 여섯 가지를 점검하고, 6-4 CDC 에 필요한 MySQL 준비를 확인합니다

✓ 코드 리뷰 — flush·poll·close·키·오류·먹등

✓ 오늘의 체크리스트

✓ 자주 겪는 오류 네 가지

✓ 6-4 를 위한 준비 확인

가져갈 것

producer.py · consumer.py · 집계 출력